Net-Worker (ProjectHub)

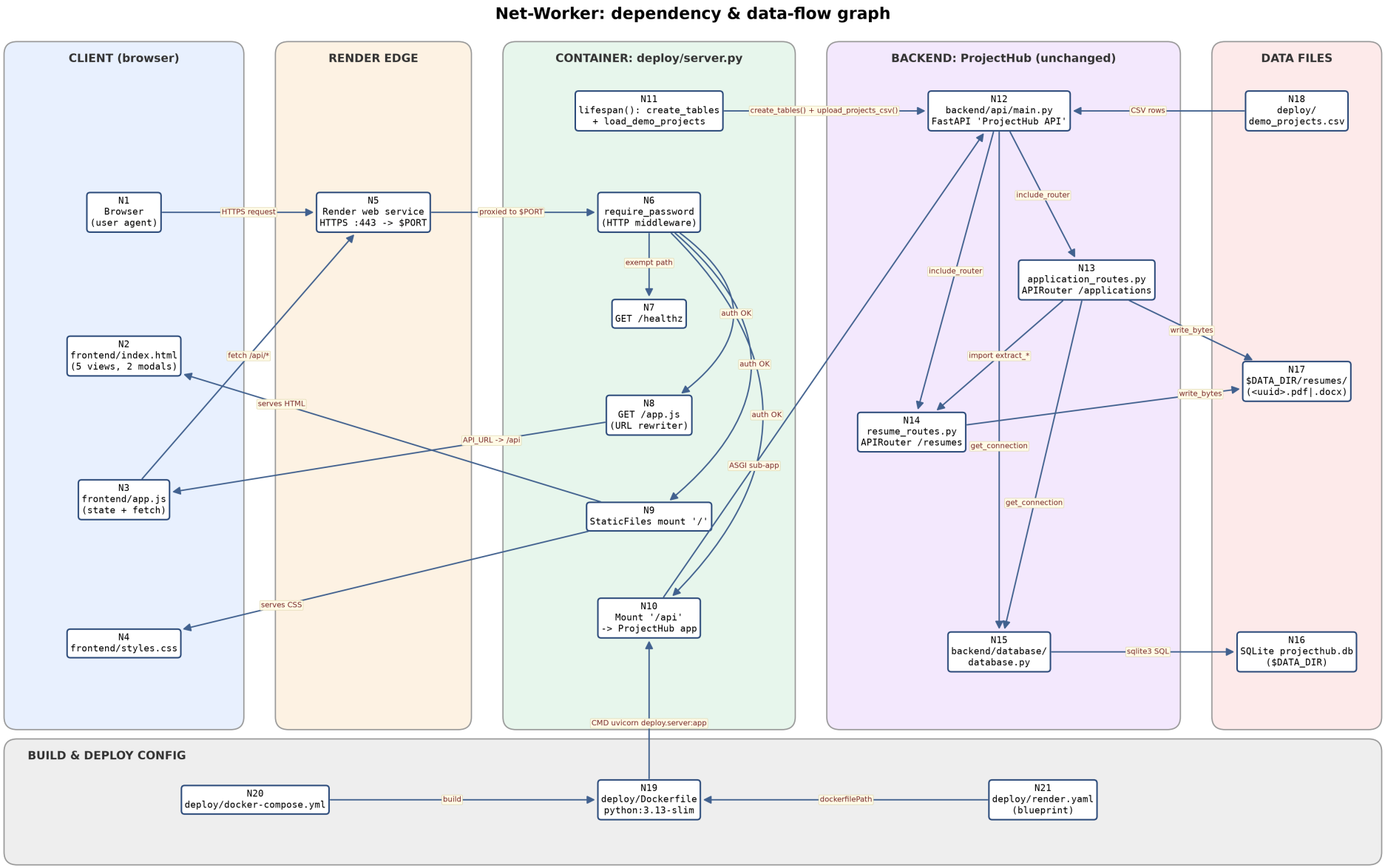
01 - Technical Architecture & Syntax

Scanned 19 tracked files (3007 lines of code), 14 HTTP routes, 4 tables. Generated 2026-11-12.

Stack: FastAPI + Starlette on uvicorn (Python 3.13), SQLite via sqlite3, vanilla JS SPA, PyMuPDF + python-docx for resume parsing, Docker image on Render free tier.

Companion: 02_Plain_English_Walkthrough.pdf uses identical section numbers and node IDs (N1-N21).

1. System graph



1. System graph - node legend

ID	Zone	Component
N1	client	Browser (user agent)
N2	client	frontend/index.html (5 views, 2 modals)
N3	client	frontend/app.js (state + fetch)
N4	client	frontend/styles.css
N5	edge	Render web service HTTPS :443 -> $PORT
N6	container	require_password (HTTP middleware)
N7	container	GET /healthz
N8	container	GET /app.js (URL rewriter)
N9	container	StaticFiles mount '/'
N10	container	Mount '/api' -> ProjectHub app
N11	container	lifespan(): create_tables + load_demo_projects
N12	backend	backend/api/main.py FastAPI 'ProjectHub API'
N13	backend	application_routes.py APIRouter /applications
N14	backend	resume_routes.py APIRouter /resumes
N15	backend	backend/database/ database.py
N16	data	SQLite projecthub.db ($DATA_DIR)
N17	data	$DATA_DIR/resumes/ (<uuid>.pdf|.docx)
N18	data	deploy/ demo_projects.csv
N19	build	deploy/Dockerfile python:3.13-slim
N20	build	deploy/docker-compose.yml
N21	build	deploy/render.yaml (blueprint)

2. Request lifecycle: Render URL -> container -> backend

- **Hop 1.** DNS resolves <service>.onrender.com; TLS terminates at Render's edge proxy.
- **Hop 2.** Proxy forwards HTTP to the container on $PORT; uvicorn hands it to deploy.server:app (ASGI).
- **Hop 3.** require_password middleware: no/invalid 'Authorization: Basic' -> 401 + WWW-Authenticate.
- **Hop 4.** Browser caches the credentials for the origin and resends them on every same-origin request.
- **Hop 5.** GET / -> StaticFiles(html=True) returns index.html; GET /app.js -> rewritten script (API_URL='/api').
- **Hop 6.** app.js DOMContentLoaded -> loadProjects() -> fetch('/api/projects').
- **Hop 7.** Mount('/api') strips the prefix; ProjectHub router matches GET /projects -> get_projects().
- **Hop 8.** get_connection() opens SQLite at DATA_DIR/projecthub.db (path patched by server.py); SELECT ... status='open'.
- **Hop 9.** Rows -> list[dict] -> JSON 200; middleware passes response back through Render to the browser.
- **Hop 10.** app.js stores it in availableProjects and re-renders cards with escapeHTML(); filters run client-side.
- **Hop 11.** Apply: multipart POST /api/applications/submit-with-resume -> validate, write <uuid>.ext, INSERT employee+application, COMMIT (rollback + unlink on error).

3. File-by-file breakdown

backend/api/application_routes.py (263 lines)

- **Imports:** backend.api.resume_routes, backend.database.database, fastapi, pathlib, sqlite3, uuid
- **Top-level functions (3):** def ensure_resume_text_column(), async def submit_application_with_resume(), def get_my_applications()
- **Route** POST /submit-with-resume -> submit_application_with_resume()
- **Route** GET /my-applications -> get_my_applications()

backend/api/main.py (547 lines)

- **Imports:** backend.api.application_routes, backend.api.resume_routes, csv, database, fastapi, fastapi.middleware.cors, io, pathlib, pydantic, re, sqlite3, sys
- **Top-level functions (12):** def startup(), def normalize_column_name(), def standardize_csv_row(), def home(), def health(), def get_projects(), def get_project(), async def upload_projects_csv(), def submit_application(), def get_application(), def get_project_applications(), def update_application_status()
- **Model** ApplicationRequest(BaseModel): project_id: int; name: str; email: EmailStr; skills: str; resume_filename: str; cover_letter: str
- **Model** ApplicationStatusUpdate(BaseModel): status: str
- **Route** GET / -> home()
- **Route** GET /health -> health()
- **Route** GET /projects -> get_projects()
- **Route** GET /projects/{project_id} -> get_project()
- **Route** POST /projects/upload-csv -> upload_projects_csv()
- **Route** POST /applications -> submit_application()
- **Route** GET /applications/{application_id} -> get_application()
- **Route** GET /projects/{project_id}/applications -> get_project_applications()
- **Route** PATCH /applications/{application_id}/status -> update_application_status()
- **Hook** @app.on_event('startup') -> startup
- **Wiring** app.include_router(resume_router, prefix='/resumes', tags=['Resumes'])
- **Wiring** app.include_router(application_router, prefix='/applications', tags=['Applications'])
- **Wiring** app.add_middleware(CORSMiddleware, allow_origins=['*'], allow_credentials=True, allow_methods=['*'], allow_headers=['*'])

backend/api/resume_routes.py (112 lines)

- **Imports:** docx, fastapi, fitz, io, pathlib, uuid
- **Top-level functions (3):** def extract_pdf_text(), def extract_docx_text(), async def upload_resume()
- **Route** POST /upload -> upload_resume()

backend/database/database.py (132 lines)

- **Imports:** pathlib, sqlite3
- **Top-level functions (2):** def get_connection(), def create_tables()

deploy/Dockerfile (33 lines)

- FROM python:3.13-slim
- ENV PYTHONDONTWRITEBYTECODE=1 \
- COPY deploy/requirements.txt deploy/requirements.txt
- RUN pip install --no-cache-dir -r deploy/requirements.txt
- COPY backend/api/*.py backend/api/
- COPY backend/database/database.py backend/database/
- COPY frontend/ frontend/
- COPY deploy/*.py deploy/*.csv deploy/
- RUN useradd --create-home appuser \
- USER appuser
- EXPOSE 8000
- CMD uvicorn deploy.server:app --host 0.0.0.0 --port $PORT

deploy/demo_projects.csv (11 lines)

- Data/config file.

deploy/docker-compose.yml (13 lines)

- Data/config file.

deploy/render.yaml (17 lines)

- runtime: docker
- plan: free
- dockerfilePath: ./deploy/Dockerfile
- healthCheckPath: /healthz

deploy/requirements.txt (6 lines)

- fastapi==0.115.6
- python-multipart==0.0.20
- email-validator==2.2.0
- pymupdf==1.25.1
- python-docx==1.1.2

deploy/server.py (149 lines)

- **Imports:** backend.api.application_routes, backend.api.main, backend.api.resume_routes, backend.database.database, base64, contextlib, fastapi, fastapi.responses, fastapi.staticfiles, io, os, pathlib, secrets, sys
- **Top-level functions (6):** async def load_demo_projects(), async def lifespan(), def password_is_correct(), async def require_password(), def healthz(), def frontend_script()
- **Route** GET /healthz -> healthz()
- **Route** GET /app.js -> frontend_script()
- **Hook** @app.middleware('http') -> require_password
- **Wiring** app.mount('/api', projecthub_api)
- **Wiring** app.mount('/', StaticFiles(directory=FRONTEND, html=True), name='frontend')

frontend/app.js (811 lines) [OVER LIMIT]

- **Module state (globals):** API_URL, employeeSkills, availableProjects, selectedProject
- **Functions (20):** escapeHTML, splitSkills, showMessage, setupNavigation, getProjects, loadProjects, loadLocationOptions, displayProjects, searchProjects, openProjectDetails, openApplicationForm, submitApplication, setupProfile, calculateMatch, showRecommendations, loadManagerProjects, loadMyApplications, uploadCSV, setupProjectActions, setupModals
- **fetch() calls:** GET /projects; GET /projects/{id}; POST /applications/submit-with-resume; GET /projects; GET /applications/my-applications; POST /projects/upload-csv
- **DOM listeners:** 14 addEventListener calls; event delegation via [data-action].

frontend/index.html (373 lines)

- **Views (<section>):** browse-view, profile-view, recommended-view, my-applications-view, manager-view
- **Modals:** project-details-modal, application-modal
- **Forms:** application-form
- Loads styles.css and app.js (defer).

frontend/styles.css (620 lines)

- Data/config file.

4. Endpoint catalogue

Public URL = what the browser calls on Render. Called = a matching fetch() exists in app.js; infra = served to the browser/Render directly.

Method	Public URL	Handler	File	Called
POST	/api/applications/submit-with-resume	submit_application_with_resume	application_routes.py	yes
GET	/api/applications/my-applications	get_my_applications	application_routes.py	yes
GET	/api/	home	main.py	NO
GET	/api/health	health	main.py	NO
GET	/api/projects	get_projects	main.py	yes
GET	/api/projects/{project_id}	get_project	main.py	yes
POST	/api/projects/upload-csv	upload_projects_csv	main.py	yes
POST	/api/applications	submit_application	main.py	NO
GET	/api/applications/{application_id}	get_application	main.py	NO
GET	/api/projects/{project_id}/applications	get_project_applications	main.py	NO
PATCH	/api/applications/{application_id}/status	update_application_status	main.py	NO
POST	/api/resumes/upload	upload_resume	resume_routes.py	NO
GET	/healthz	healthz	server.py	infra
GET	/app.js	frontend_script	server.py	infra

5. Data models

Table projects (referenced in 8 query/queries outside the DDL)

- id INTEGER PRIMARY KEY AUTOINCREMENT
- role_id TEXT UNIQUE
- title TEXT NOT NULL
- client_project TEXT
- required_skills TEXT
- nice_to_have_skills TEXT
- seniority TEXT
- location TEXT
- start_date TEXT
- status TEXT DEFAULT 'Open'
- description TEXT
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Table employees (referenced in 9 query/queries outside the DDL)

- id INTEGER PRIMARY KEY AUTOINCREMENT
- name TEXT
- email TEXT UNIQUE
- skills TEXT
- resume_filename TEXT
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- resume_text TEXT (added at runtime by ALTER TABLE)

Table matches (referenced in 0 query/queries outside the DDL)

- id INTEGER PRIMARY KEY AUTOINCREMENT
- employee_id INTEGER
- project_id INTEGER
- match_score REAL

- matched_skills TEXT
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- (constraint) FOREIGN KEY (employee_id) REFERENCES employees(id)
- (constraint) FOREIGN KEY (project_id) REFERENCES projects(id)

Table applications (referenced in 7 query/queries outside the DDL)

- id INTEGER PRIMARY KEY AUTOINCREMENT
- employee_id INTEGER NOT NULL
- project_id INTEGER NOT NULL
- application_status TEXT DEFAULT 'Submitted'
- resume_filename TEXT
- cover_letter TEXT
- submitted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- (constraint) FOREIGN KEY (employee_id) REFERENCES employees(id)
- (constraint) FOREIGN KEY (project_id) REFERENCES projects(id)
- (constraint) UNIQUE(employee_id, project_id)

Pydantic request models

- ApplicationRequest(BaseModel): project_id: int; name: str; email: EmailStr; skills: str; resume_filename: str; cover_letter: str
- ApplicationStatusUpdate(BaseModel): status: str

6. State management

- Client globals in app.js: employeeSkills[], availableProjects[], selectedProject - in-memory only, lost on reload.
- Recommendation score = |required n employeeSkills| / |required| computed in the browser (calculateMatch); the 'matches' table is never written.
- Server is stateless per request: each handler opens and closes its own sqlite3 connection.
- Durable state: SQLite file + resume files under DATA_DIR. Render free tier has an ephemeral disk -> reset on deploy/restart/sleep.
- Identity: none in the app; 'My Applications' trusts any email in the query string. Basic-auth is one shared team credential.

7. Files over 800 lines

File	Lines	Status
frontend/app.js	811	OVER - plan refactor
frontend/styles.css	620	ok
backend/api/main.py	547	ok
frontend/index.html	373	ok
backend/api/application_routes.py	263	ok
deploy/docs/round1-design.md	166	ok
deploy/server.py	149	ok
backend/database/database.py	132	ok
backend/api/resume_routes.py	112	ok
deploy/Dockerfile	33	ok
deploy/README.md	31	ok
deploy/render.yaml	17	ok

File	Lines	Status
.gitignore	14	ok
deploy/docker-compose.yml	13	ok
deploy/demo_projects.csv	11	ok
deploy/Dockerfile.dockerignore	9	ok
deploy/requirements.txt	6	ok
README.md	1	ok

Refactor proposal for app.js: split into ES modules - api.js (fetch wrappers + API_URL), state.js (the three globals), views/browse.js, views/recommend.js, views/applications.js, views/manager.js, ui.js (escapeHTML, showMessage, modals). Keep the API_URL literal in one place so deploy/server.py's swap keeps working.

8. Risks & observations

- Two application write paths: POST /applications (JSON, main.py) and POST /applications/submit-with-resume (multipart). Only the latter is called; logic is duplicated.
- database.py is imported twice (as 'database' via sys.path and as 'backend.database.database'): two module objects; server.py must patch both.
- CORSMiddleware allow_origins=['*'] with allow_credentials=True; harmless behind same-origin /api but permissive if main.py is run alone.
- @app.on_event('startup') is deprecated in FastAPI; it is also skipped when mounted, which is why server.py has its own lifespan.
- Profile 'resume-file' input is never uploaded; POST /resumes/upload stores a file but nothing links it to an employee.
- GET /applications/{application_id} relies on the router being included first so /my-applications wins the match.
- Real projecthub.db and resumes are committed to git (outside the image thanks to the Dockerfile's explicit COPY list).